

CSCE 20104 – Programming Project 4
Due Date – Wednesday 03/11/2026 at 11:59 pm

1. Problem Statement:

The goal of this assignment is to give students experience with recursive binary search and its application for document analysis. In particular, your task is to create two programs that will enable you to create a very basic method for doing a lexical comparison of multiple documents (much like Google search). Detailed instructions will be given below.



Your first program called “goo” will preprocess eBooks from <https://www.gutenberg.org> to extract information about the most common words in the eBook and save this in a word frequency file. This program must use recursive binary search to look up each of the words in the eBook and discard any words that are among the 100 most common English words (e.g., “the”, “be”, “to”, “of”, “and”) since they provide very little information about the eBook.

Your second program called “googoo” will read two eBook word frequency files and calculate the lexical similarity of the two eBooks. This should be done by counting the words the two eBooks have in common. For example, two science fiction books may have words like “space”, “rocket”, “extraterrestrial” in common and have a high similarity score. On the other hand, a children’s book and a romance novel may have a low similarity score because they have very few words in common.

For this project, you will be given specific instructions on how and when generative AI may be used in the software development process. Other uses of generative AI are NOT allowed. Remember to record all of your AI prompts in a separate file to be included with your project report on the due date.

2. Design:

Task 1: Creating the “goo” program:

- A) Download a collection of 10 eBooks from <https://www.gutenberg.org>. When you go to this website you will see they have copies of over 75,000 free eBooks. You can select any 10 eBooks you wish by selecting from the most frequently downloaded eBooks or by searching different categories (science fiction, romance, classic literature, etc.). Save the “Plain Text UTF-8” version of your 10 eBooks to your local machine.
- B) Create a C++ function that reads in an ASCII file one word at a time and stores these words in a vector of strings. Your function should remove all characters that are not

between [a..z] or [A..Z] from the input string before adding it to the end of the vector. You can use the “isalpha” function to identify the characters to keep. You ARE allowed to use generative AI to help implement this function.

- C) Use the C++ vector sort method from the std library to sort your vector of words in ascending order. You are encouraged to use Google to search for the official C++ documentation of this function. As you do this, you will see that Google now has an AI mode that will give you their explanation of this function and sample code showing how to use the sort function. You ARE allowed to use sample code from the C++ documentation website or from Google in your program.
- D) Now that you have sorted all of the words in the eBook in alphabetical order, you can calculate the frequency of each word and write this to an output file. Loop over your sorted vector and count how many times each word occurs. If the count is greater than one, write the count and the word on one line in an ASCII output file. If you take a look at this file you will see that words like “the” “be” “to” “of” and “and” have huge word frequencies which really don’t tell us anything interesting about the eBook being processed.
- E) Ask Google or your favorite generative AI to create an ASCII file containing the top 100 most commonly used words in English (this list will vary slightly depending on your source). Extend your program to read these 100 words from an ASCII file into a vector and sort them alphabetically. You don’t need to store word rank information.
- F) Implement a recursive binary search function that takes in a string and returns true if the string appears in the vector of the top 100 words and false otherwise. Use your search function in your program to exclude all words in the top 100 from your output file. You can either exclude words as are read from the input file, or you can exclude words after you calculate the word frequency and before you write a word and its frequency to the output file. You ARE allowed to use sample code from the CSCE 2014 website when implementing the binary search function.
- G) Finally, use your “goo” program to calculate the word frequencies of all 10 of your eBooks. Be sure to give the eBooks and their associated word frequency files sensible file names so you can keep everything organized.

Task 2: Creating the “googoo” program:

- A) Write a C++ function that reads word frequency information from an input file and stores this information in two vectors. As you do this, calculate and print the number of unique words that appear in the input file and the total number of words in the eBook based on the word frequencies.
- B) Create a C++ program called “googoo” that prompts the user for the names of two eBooks and reads their corresponding word frequencies into vectors. Using this information, count how many words appear in both of the eBooks. There are several

ways you could implement this. For example, you could scan the vector of words from one eBook and compare each word to all the other words in the other eBook vector one at a time using linear or binary search. Alternatively, you could scan both vectors of words simultaneously and count the number of matching words. You are welcome to implement any method you wish, and describe your choice in the project report.

- C) Since the number of unique words in a book may vary significantly, you should to normalize the count of matching words by dividing by the average number of unique words in the two eBooks being processed. Once you have done this, print the eBook lexical similarity as a value between 0% and 100%.
- D) Finally, use your “googoo” program to compare one of your eBooks to the remaining nine eBooks and create a table of eBook similarity scores to include in your project report. Look at your most similar eBooks and least similar eBooks. Do you think the results make sense based on your knowledge of these books?

3. Implementation:

For this project you will be creating two C++ programs “goo” and “googoo”. If you use OnlineGDB or VS Code, you can create these programs using the methods described in the lab. To compile your programs on turing use the commands:

- g++ -Wall goo.cpp -o goo.exe.
- g++ -Wall googoo.cpp -o googoo.exe.

Once you have chosen the methods for each class it is a good idea to start with "skeleton methods" to get something to compile, and then add the desired code to each method incrementally writing comments, adding code, compiling, debugging, a little bit at a time. Once you have the methods implemented, it should be fairly simple to create a main program that calls these methods to complete your project.

Remember to use good programming style when creating your program; clear names for variables and constants, proper indenting for loops and conditionals, clear comments, etc. Also, be sure to save backup copies of your program somewhere safe. Otherwise, you may end up retyping your whole program if something goes wrong.

4. Testing:

Test your program to check that it operates correctly for all of the requirements listed above. See what happens if the user does not follow your instructions, or if they type invalid data. Try your program with several input values, and save your testing output in text files or as screenshots for inclusion in your project report.

Once your program is working properly, you need to demonstrate correctness on the department Linux server turing.csce.uark.edu. To do this, you need to copy your source

code and data files to turing and compile and execute your program while running “script” as shown below.

```
script testing.txt
g++ -Wall goo.cpp -o goo.exe
g++ -Wall googoo.cpp -o googoo.exe
./goo.exe
(type your inputs)
./googoo.exe
(type your inputs)
exit
```

If you are unable to login to turing to create the testing.txt file, you can use your personal computer instead. If you have a Mac, you can open a terminal window to execute the commands above. If you have a PC, you can install the Windows Subsystem for Linux (WSL) and run these commands in a terminal window.

5. Documentation:

When you have completed your program, write a short report using the “Programming Project Report Template” describing what the project objectives were, what you did, and the status of the program. The report template has a number of point form questions in each section. You should remove these questions from the report and replace them with your answers. Save this project report in a separate document to be submitted electronically.

6. Project Submission:

In this class, we will be using electronic project submission to make sure that all students hand their programming projects and labs on time, and to perform automatic plagiarism analysis of all programs that are submitted.

When you have completed the tasks above, create a folder on your computer called “your_last_name” and copy all of your C++ source code, input/output files, your testing.txt file, prompts.docx file, and project report into this folder. Then compress this file folder to create “your_last_name.zip”. Upload this zip file into Blackboard using the project upload link. We will download and decompress your zip file, and compile and run your code to verify correctness.

The dates on your electronic submission will be used to verify that you met the due date above. All late projects will receive reduced credit:

- 25% off if less than 1 day late,
- 50% off if less than 2 days late,
- 75% off if less than 3 days late,
- no credit if more than 3 days late.

You will receive partial credit for all programs that compile, even if they do not meet all program requirements, so handing projects in on time is highly recommended.

7. Academic Honesty Statement:

Students are expected to submit their own work on all programming projects, unless group projects have been explicitly assigned.

- Students are NOT allowed to distribute code to each other or copy code from another individual or website.
- Students are NOT allowed to use generative AI to create code UNLESS specific instructions on how to use generative AI are included in the project description.
- If generative AI is explicitly allowed for this project, all AI prompts should be saved in "prompts.docx" to be submitted with the rest of the project.
- Students ARE allowed to use any of the source code examples on the class website, or in the textbook in this project.
- Students ARE allowed to use online C++ documentation websites like <https://cplusplus.com/> for reference materials and C++ syntax examples.
- Students ARE allowed to ask the instructor and/or GTAs for assistance during lecture or office hours.

This course will be using highly effective program comparison software to detect plagiarism and unauthorized use of generative AI. Please do not be tempted to copy code from another student or from unauthorized sources. Violations of the policies above will be reported to the Academic Initiatives and Integrity office <https://honesty.uark.edu/> and may result in a ZERO on the programming project, an F in the class, or suspension from the university, depending on the severity of the violation and any history of prior violations.